UNIFIED ENGINEERING WORKBENCH

TECHNICAL REPORT | R&D; INTERNSHIP 2026–2027

PROJECT	Unified Engineering Workbench
CANDIDATE ID	RA2311003012331 (verify against final archive name)
IMPLEMENTATION	C++17 computational backend with React / TypeScript frontend
REPORT SCOPE	Design, numerical verification, parallel execution, embedded integration, testing and limitations

Executive Summary

The Unified Engineering Workbench is designed as a single interface for engineering data inspection and computational experiments. The assignment calls for ASCII STL visualization, ANSYS-style nodal-temperature CSV visualization, a steady-state one-dimensional thermal finite-element solver, reproducible serial/parallel thermal workloads, and a host-connected embedded-system design represented with C++ mock adapters. The supplied repository structure also identifies frontend views, C++ modules, datasets, results, tests, packaging scripts and supporting documentation.

Project objectives

1. Import and validate engineering geometry and temperature data.
2. Solve and verify the specified 1D steady-state thermal problem using finite elements.
3. Compare serial and CPU-parallel batches using reproducible measurements.
4. Represent microcontroller, FPGA and JTAG adapter integration safely through clearly labelled mocks.
5. Demonstrate reliable startup, understandable results and evidence-backed testing.

Key engineering principle

The report separates assignment requirements from claims that still need measured evidence. No benchmark numbers, test pass counts or clean-machine launch outcomes are invented. Populate those items from the actual CSVs, logs and Windows test run before submitting.

Document map

Page 2: architecture and data flow. Page 3: algorithms and verification method. Page 4: results and performance evidence. Page 5: testing and embedded design. Page 6: deployment, limitations and references.

1. System Architecture and Data Flow

The assignment permits a web or desktop interface, but requires backend logic, numerical computation, file processing and concurrency to use C++17 or later. The diagram below summarizes the intended module boundaries from the project structure and assignment requirements; confirm exact API routes and runtime behavior against the code.

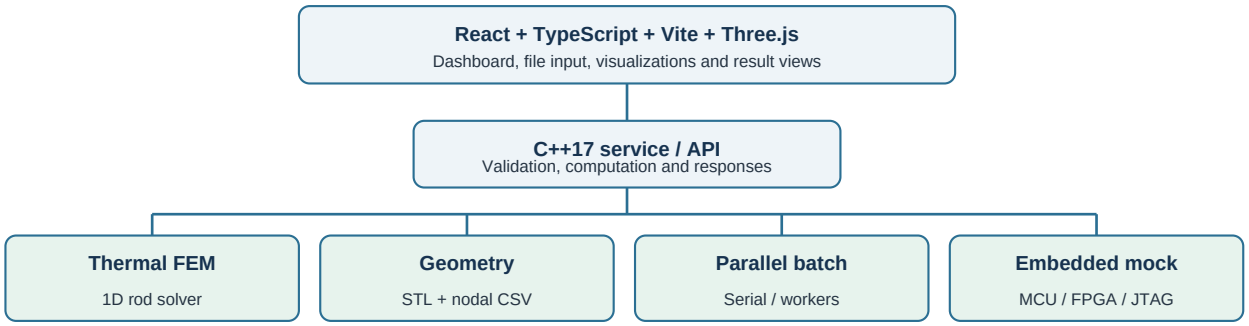

Figure 1. High-level component view; validate actual connections and endpoints against the implementation.

Component responsibilities

Layer	Responsibility	Evidence / location
Frontend	Unified navigation, file selection, 3D interaction, status, plots and export.	frontend/src/; frontend/dist/
C++ backend	Parsing, validation, numerical computation, parallel work and mock-device operations.	src/main.cpp; src/
Geometry input	ASCII STL triangles and nodal temperature CSV point cloud.	data/; STL parser and CSV reader
Thermal module	Linear two-node element assembly, boundary conditions, solution and error analysis.	src/core/thermal_solver.hpp; results/
Performance module	Fixed-seed independent cases; serial and bounded worker-pool execution.	src/performance/; benchmark CSVs
Embedded mock layer	Discovery, capabilities, connect/disconnect, telemetry and recoverable failure simulation.	src/hardware/; Hardware view
Verification / delivery	Automated tests, raw outputs, build scripts, launch package and documentation.	tests/; results/; scripts/; docs/

Data flow

A typical workflow is: user selects a file or experiment settings → frontend validates basic form inputs → C++ backend performs authoritative file validation and computation → structured results are returned → frontend renders geometry, temperature values or plots → raw numerical output is exported for inspection. The backend must remain the source of truth for parsing and numerical calculations; rendering libraries should not replace required C++ file processing.

Design requirements to verify

- Relative paths work after ZIP extraction.
- Invalid, empty, non-finite or malformed input produces actionable errors.
- UI status remains responsive during long computations, with safe completion or cancellation.
- Presentation, transport and computational code remain separated.

2. Algorithms and Numerical Verification

2.1 Geometry and temperature-data visualization

The required geometry input is an ASCII STL file. The viewer should support rotate, pan, zoom and fit-to-view. The temperature input is a CSV with columns node_id, x_m, y_m, z_m and temperature_C; the UI should display a coloured 3D point cloud, temperature legend, units and node inspection. Mesh-to-result registration is not required. The C++ parser should reject malformed records and non-finite values and explain empty-file errors. Document file-size limits that were actually tested.

2.2 Steady-state 1D thermal FEM

The specified rod has L = 1 m, A = 1×10■■ m², k = 10 W/(m·K), uniform volumetric heat generation q = 10,000 W/m³, T(0) = 100 °C and T(L) = 20 °C. Lateral surfaces are insulated and material properties are constant. The governing equation is –d/dx(kA dT/dx) = qA. For each two-node linear element, assemble the element stiffness and load contributions into the global system, impose both prescribed endpoint temperatures correctly, and solve for nodal temperatures.

Analytical check and convergence

The assignment provides the reference solution T_exact(x) = 100 – 80x/L + [q/(2k)]x(L–x). Evaluate the FE solution at element midpoints by interpolation and report maximum absolute midpoint error for N = 10, 20, 40 and 80 uniform elements. Plot temperature against x, label units, export the underlying data and show how error changes with mesh refinement. Do not report nodal-only error as a substitute for the required midpoint error.

2.3 Serial and parallel execution

Generate independent thermal cases using a fixed seed, varied positive k and q, configurable case count and mesh size. Run the same cases with the same numerical method in serial and with 1, 2 and up to 4 workers, capped by available logical CPUs. Store results in deterministic input order and verify serial/parallel agreement with maximum absolute temperature difference ≤ 1×10■■ °C.

Performance metrics

Measure compute-only wall-clock time, excluding file loading, rendering and compilation. Use a warm-up and five measured repetitions; report median time. For p workers, speedup = T■/T■ and efficiency = (T■/T■)/p. Record CPU model, physical/logical core counts, RAM, compiler, optimization flags, workload, and separately report launch-to-ready time. Explain overhead and disclose if the machine has only one logical CPU.

2.4 Neural-network backpropagation

The handwritten appendix must show the specified forward pass, chain-rule derivatives for w, b, v and c, a simultaneous update using the original gradients, and a gradient-direction explanation. The matching C++ implementation must avoid automatic differentiation, compare analytical gradients with central finite differences at ε = 10■■ with absolute error ≤ 10■■, and plot loss over 100 subsequent steps at learning rate 0.1.

3. Experimental Results and Evidence

This section is intentionally evidence-led. The assignment asks for measured FEA convergence and reproducible parallel timing. Insert the actual values produced by the submitted implementation; filenames or screenshots alone do not prove numerical correctness.

3.1 Thermal FEA results

Elements (N)	Max midpoint absolute error (°C)	CSV / plot reference
10	[Insert measured value]	[results file]
20	[Insert measured value]	[results file]
40	[Insert measured value]	[results file]
80	[Insert measured value]	[results file]

Interpretation to write after checking the data: describe whether the midpoint error decreases with refinement, whether the observed trend matches expectations, and any debugging issue discovered. Include a plot of the FE and analytical temperature profiles with x in metres and temperature in °C.

3.2 Parallel benchmark results

Workers	Median compute time (s)	Speedup	Efficiency
1	[T1]	1.00	100%
2	[T2]	T1 / T2	(T1 / T2) / 2
3	[T3]	T1 / T3	(T1 / T3) / 3
4 or CPU cap	[Tp]	T1 / Tp	(T1 / Tp) / p

Record the machine/compiler/workload details and five raw timing samples for each worker count. Explain measured overhead and scaling; do not promise speedup if measurements do not show it. If one logical CPU is available, demonstrate the parallel code path and disclose that speedup could not be assessed.

3.3 Backpropagation and gradient check

Report the maximum absolute difference between each analytical gradient and its central finite-difference estimate, plus the tolerance used. Include the loss curve for 100 training steps and a short interpretation grounded in the plotted values.

3.4 Evidence inventory

Attach or reference the raw CSV files and labelled plots for FEA convergence, performance timings and neural loss. Each figure should have a clear title, labelled axes, units and a caption. Keep data and plotting scripts together so results can be regenerated.

Submission action required: replace every bracketed placeholder with measured data or explicitly mark the corresponding experiment as not yet completed.

4. Testing, Embedded Design and Reliability

4.1 Verification matrix

Test area	Required verification	Evidence to record
STL parser	Valid ASCII STL, malformed facets, empty file, invalid/non-finite values and size limit.	Test cases, expected errors and results.
Temperature CSV	Required schema, units, invalid numbers, empty input and node inspection.	Fixture files and test logs.
Thermal solver	Boundary conditions, analytical comparison, midpoint error for four meshes.	Raw CSV, convergence plot and tolerance.
Parallel execution	Identical cases, deterministic order and ≤ 1e-8 °C serial/parallel difference.	Comparison log and timing CSV.
Backpropagation	Four finite-difference gradient checks and 100-step loss curve.	Gradient errors, loss data and plot.
API / UI	Successful requests, invalid inputs, error display and responsive status.	Test logs and genuine screenshots.
Packaging	Fresh Windows 11 x64 extraction, double-click launch, close and relaunch.	Machine details, steps and observed outcomes.

4.2 Embedded-system design

The assignment asks for a modular host-connected design containing one microcontroller board, one FPGA board and one JTAG debug adapter. Provide a labelled block diagram and a concise interface/pin-voltage table covering power, common ground, voltage domains, level shifting, connectors, host communication, JTAG signal ownership and device-specific pinout assumptions. A JTAG adapter is a debug interface, not a generic interchangeable compute board; do not assume connectors, voltages or protocols are shared across devices.

The C++ mock adapters should demonstrate discovery, capability reporting, connect/disconnect and one telemetry message for all three device types in the UI, plus one recoverable failure such as a timeout. Clearly label simulated behavior and state that physical wiring, PCB fabrication, firmware deployment and real FPGA programming are outside the assignment scope.

4.3 Limitations and risk controls

- Mocked device interactions are not evidence of physical hardware operation.
- Numerical accuracy depends on correct assembly, boundary conditions, units and implementation.
- Benchmark results depend on workload, hardware, compiler settings and system load.
- File-size limits and supported encodings must reflect tests actually performed.
- UI screenshots demonstrate appearance, not correctness; pair them with logs and numerical checks.

Testing status must be factual: provide exact test command, environment and passed/failed/skipped counts from the latest run. Do not infer counts from source-file names.

5. Deployment, Conclusion and References

5.1 One-click deployment

The assignment explicitly targets Windows 11 x64. After ZIP extraction, double-clicking Launch.exe must open the working application without terminal commands, PowerShell, administrator rights, package installation or a first-run build. Runtime dependencies must be bundled lawfully. The standalone launcher must not require the editor or internet. Also include a VS Code or Antigravity run configuration that starts the same packaged application in one action after the documented configuration is selected; document any editor-extension prerequisite. Provide CMake source-build instructions separately.

Clean-machine release checklist

1. Extract the final CandidateID_Workbench.zip into a new folder.
2. Double-click Launch.exe and record launch-to-ready time.
3. Confirm the UI and C++ backend start, then exercise all required views.
4. Test valid and invalid STL/CSV inputs and one recoverable mock-device failure.
5. Confirm no internet, admin rights, terminal or first-run build is required.
6. Close the application and verify processes exit safely; relaunch once.
7. Verify relative paths, dependency/version information, editor configuration and SHA-256 checksum.

5.2 Conclusion

The workbench is intended to unite geometry and temperature-data visualization, thermal finite-element analysis, multicore execution and embedded integration design in one coherent interface. A strong submission must demonstrate more than a polished UI: it must show correct numerical formulation, analytical verification, reproducible timing, robust file handling, explicit mock-device boundaries and a successful standalone launch. The final conclusion should be updated after reviewing the measured results and clean-machine test evidence.

5.3 References and project evidence

Cite the exact sources used in the implementation. Appropriate categories include the C++17 language/library documentation, CMake documentation, official React/TypeScript/Vite/Three.js documentation, a finite-element methods reference supporting the chosen formulation, and the specific STL-format reference consulted. Add complete titles, authors or organizations, publication/version details, URLs and access dates where applicable. Do not list sources that were not actually consulted.

Related submission evidence

Include the README, dependency/version information, data provenance and schema notes, test outputs, results CSVs and plots, AI-use log, ownership statement, meaningful Git history and a 3–5 minute demonstration video or timestamped screenshot walkthrough with logs. The handwritten appendix is separate and limited to two pages; the technical report is limited to six pages excluding that appendix.

FINAL REVIEW: verify every technical claim against source code and raw outputs. Replace all bracketed fields, add actual references, and confirm the PDF remains six pages before packaging.